MLOPS - LLMOPS SCREENSHOTS

Index

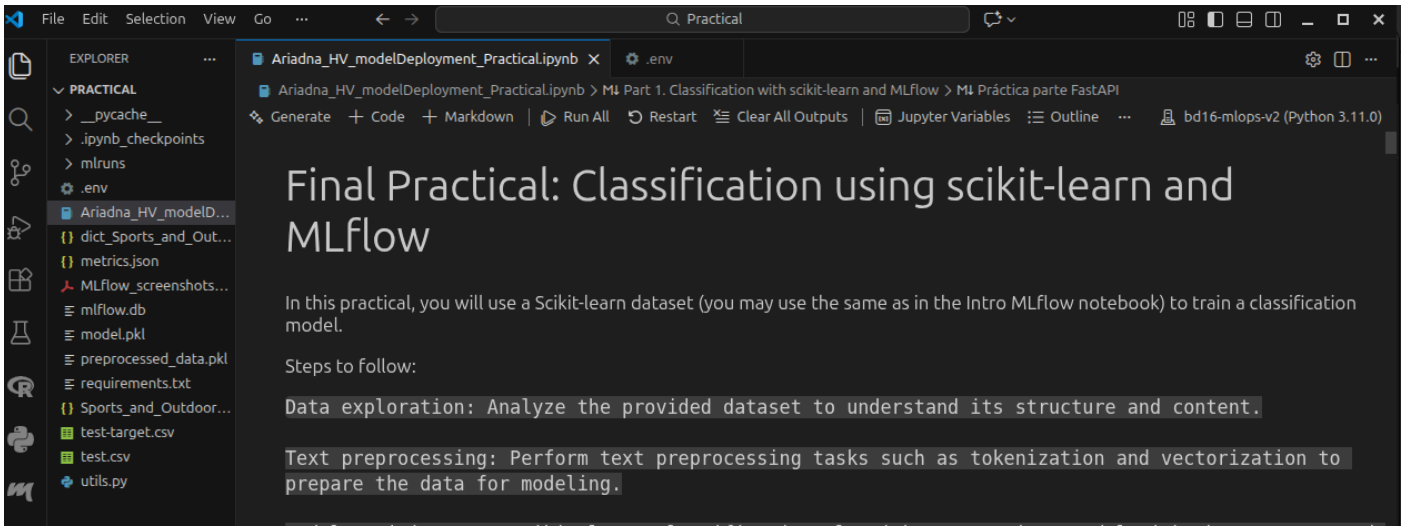
0. Set up of practical . . . . . p. 1

1. MLflow screenshots . . . . . p. 2
Connecting MLflow with G. Generative AI and LangChain . . . . . p. 6

2. FastAPI screenshots . . . . . p. 14
Pipelines (Transformers) . . . . . p. 22

0. Set up of practical

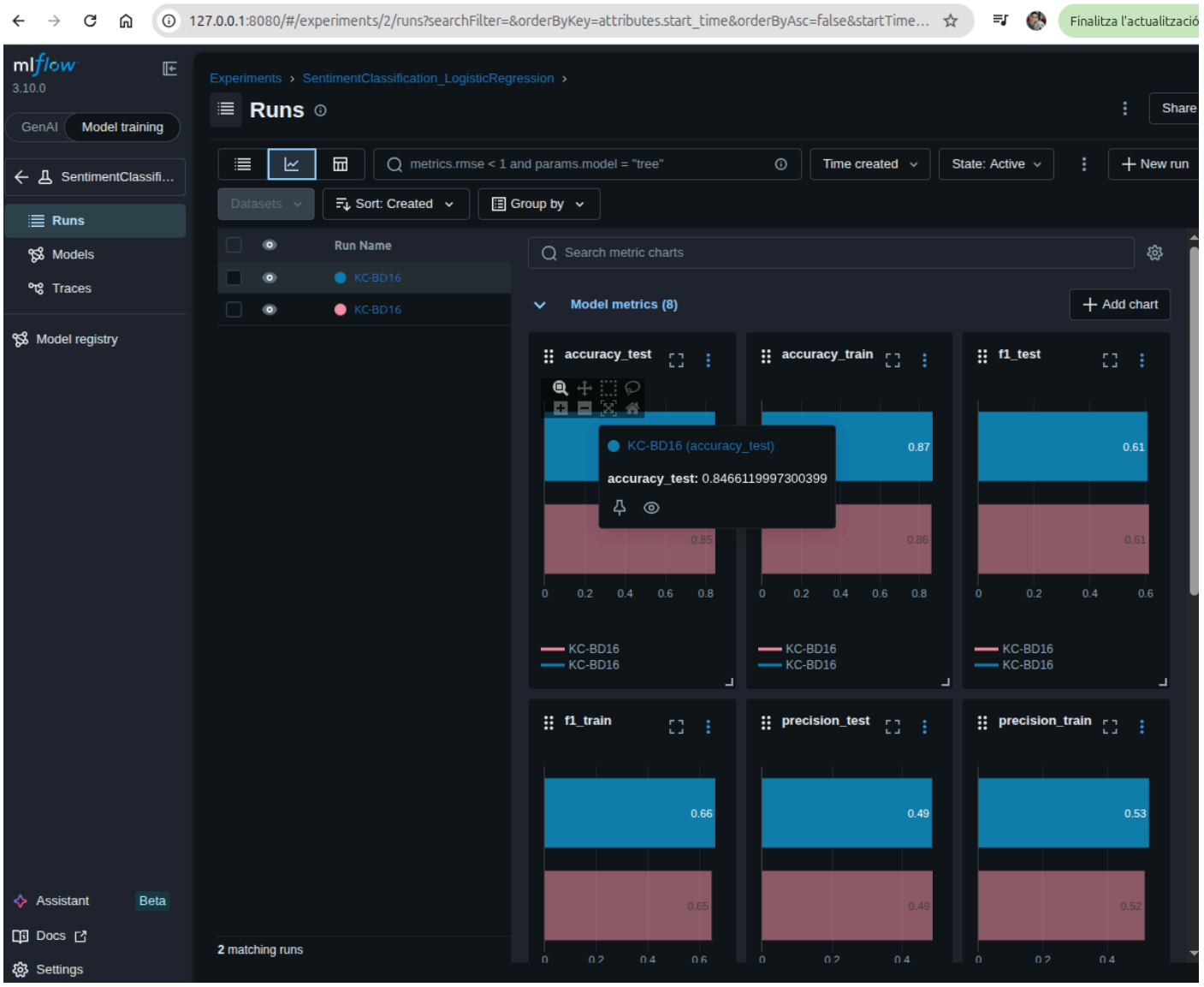
This is a screenshot of my VisualStudioCode project, with the different files (visible on the left) that have been used in the code or generated by the code.



1. MLflow screenshots

This document contains a few screenshots of the interaction between my code (in the notebook ***Ariadna_HV_modelDeployment_Practical.ipynb***) with the UI of MLflow.

Chart view - comparison between the 2 runs. You may also click on the box to see the numeric value.



Side-by-side comparison between the two models:

127.0.0.1:8080/#/compare-runs?runs=["9d5042c391834d6f9d24b193ddb2b46b","59294b373404489c8d96632318c...

Run details

Run ID:	9d5042c391834d6f9d24b193ddb2b46b	59294b373404489c8d96632318c6feb5
Run Name:	KC-BD16	KC-BD16
Start Time:	03/03/2026, 02:48:18 AM	03/03/2026, 01:55:58 AM
End Time:	03/03/2026, 02:48:25 AM	03/03/2026, 01:56:03 AM
Duration:	6.7s	5.5s

Parameters

Show diff only

classifier__C	5	1
classifier__penalty		l2
param_classifier	LogisticRegression(C=5, class_weight='balanced', max_iter=1000)	LogisticRegression(C=1, class_weight='balanced', max_iter=1000, penalty='l2')
param_classifier__C	5	1
param_classifier__penalty	deprecated	l2
param_steps	[('preprocessor', ColumnTransformer(transformers=[('tfidf', TfidfVectorizer(max_df=0.95, max_features=10000, min_df=10, ngram_range=(1, 3)), 'processedText'), ('length', StandardScaler(), ['reviewLength']])]), ('classifier', LogisticRegression(C=5, class_weight='balanced', max_iter=1000))]	[('preprocessor', ColumnTransformer(transformers=[('tfidf', TfidfVectorizer(max_df=0.95, max_features=10000, min_df=10, ngram_range=(1, 3)), 'processedText'), ('length', StandardScaler(), ['reviewLength']])]), ('classifier', LogisticRegression(C=1, class_weight='balanced', max_iter=1000, penalty='l2'))]

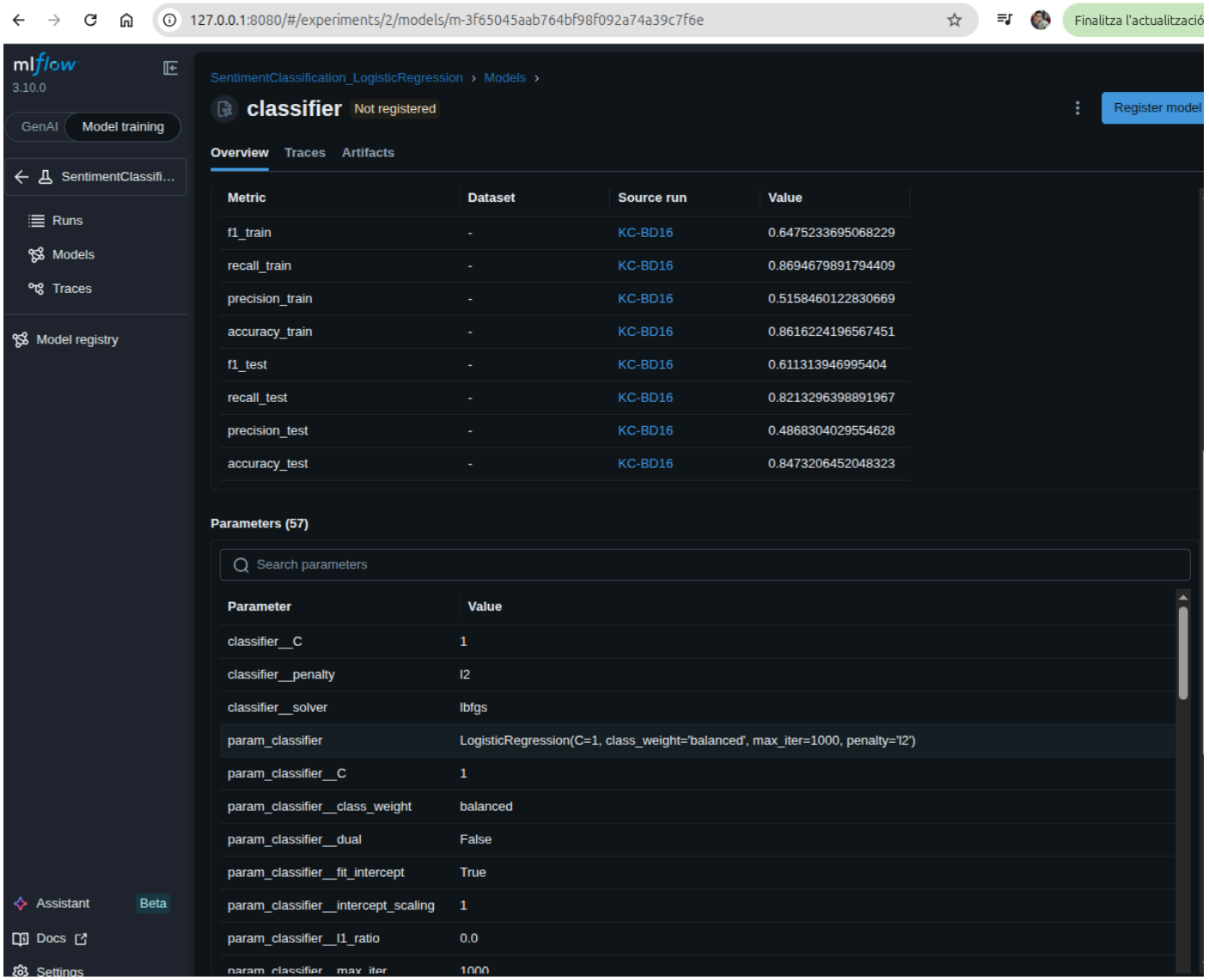
Metrics

Metrics

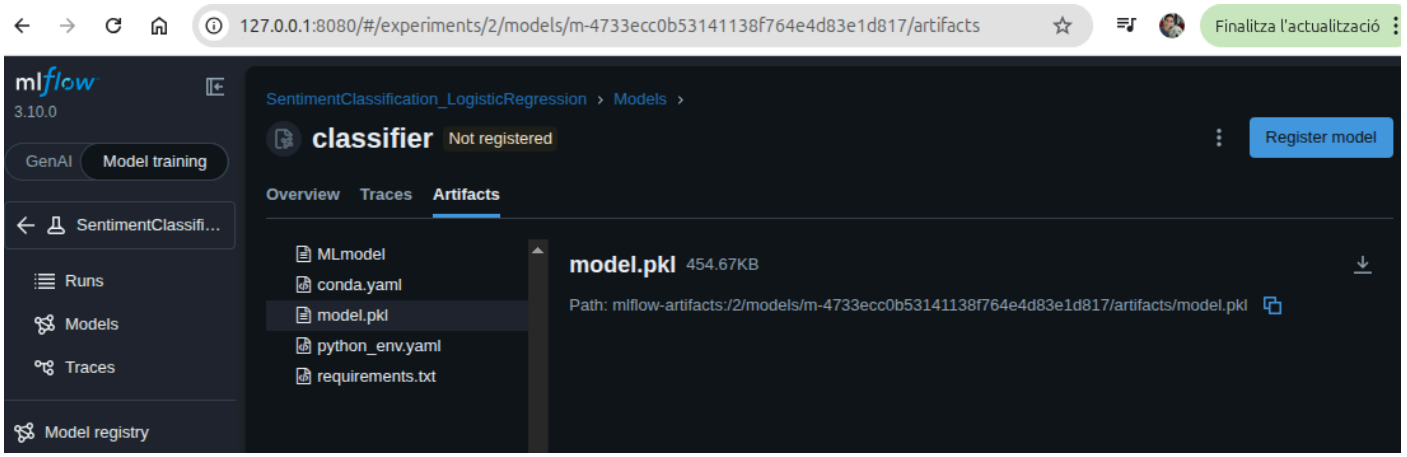
Show diff only

accuracy_test	0.847	0.847
accuracy_train	0.868	0.862
f1_test	0.606	0.611
f1_train	0.662	0.648
precision_test	0.485	0.487
precision_train	0.529	0.516

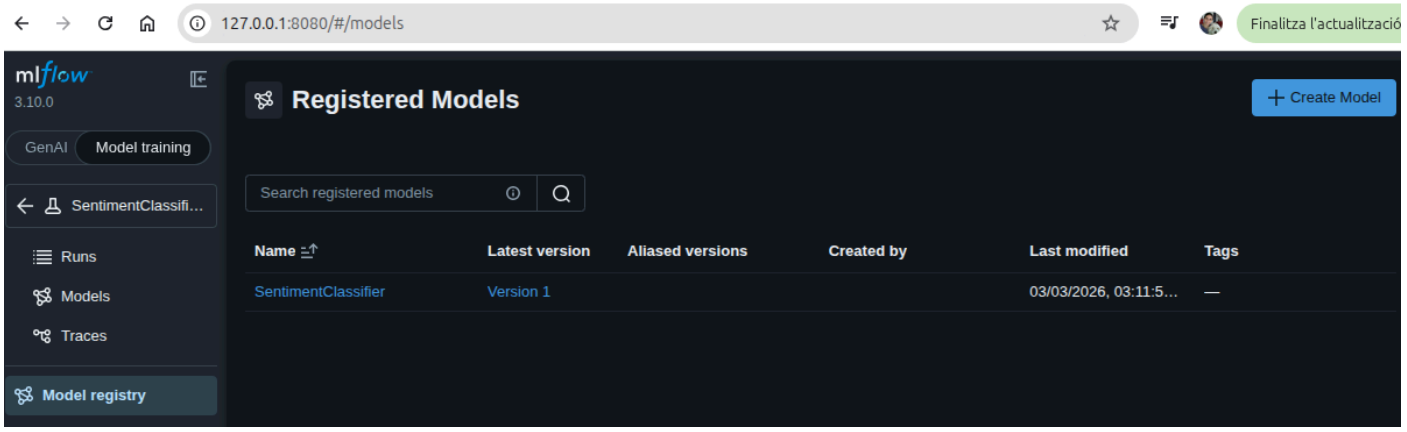
Best model, though it may overfit the training data slightly more than the other model:

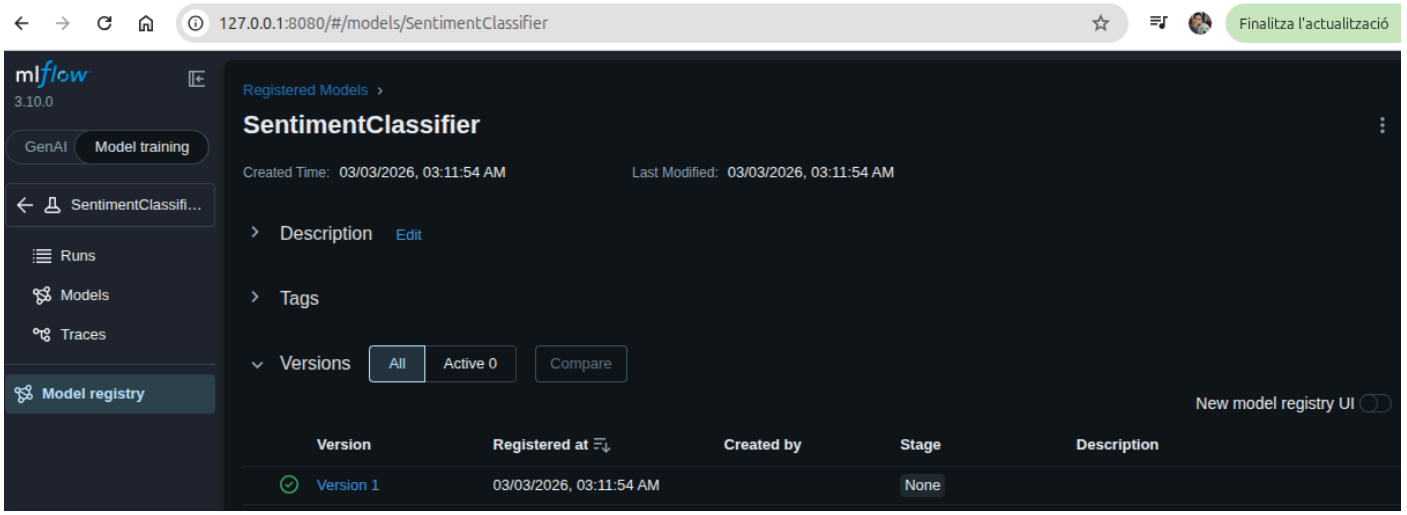


We can also see the **artifacts** associated with the model:

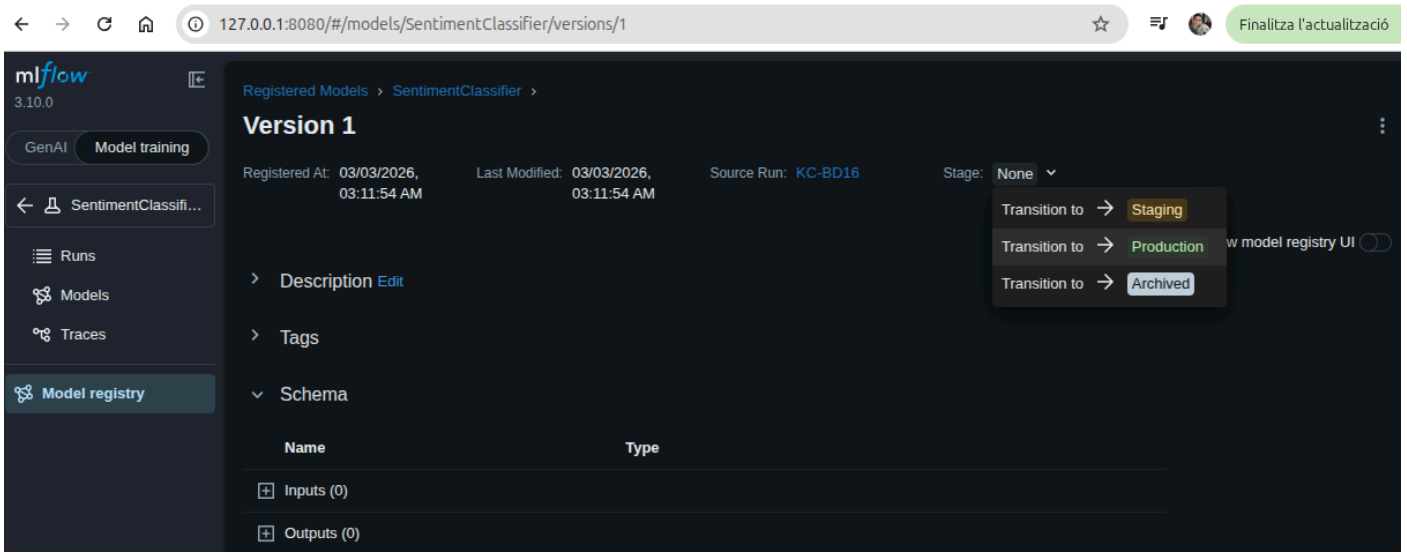


Let's register it:

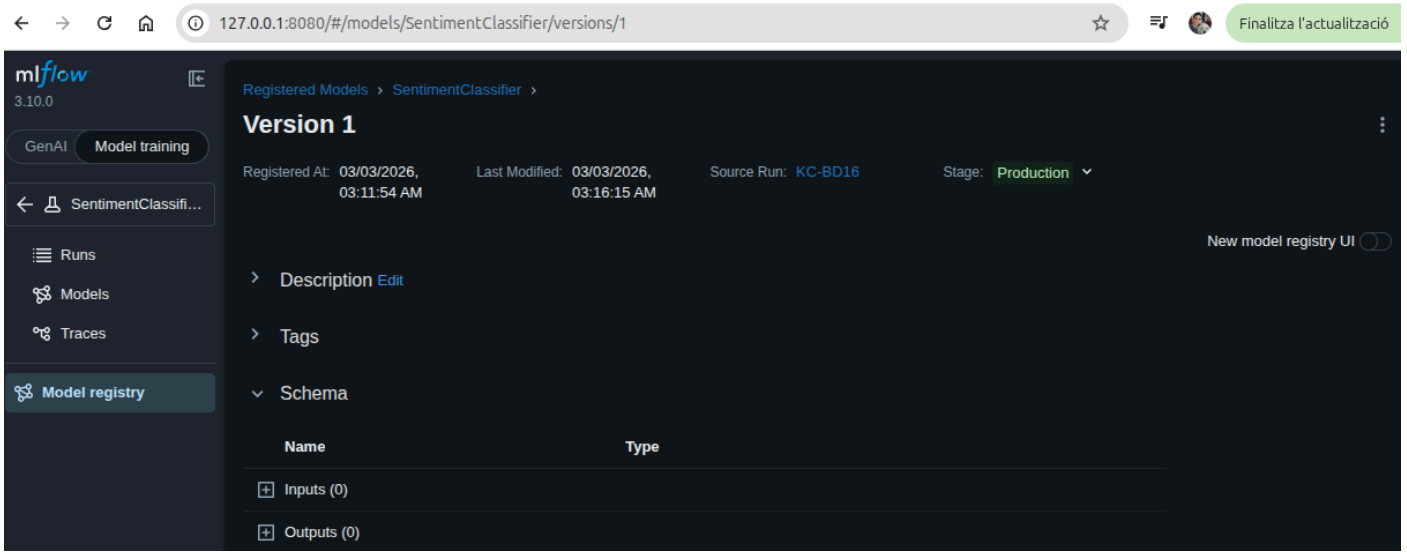




And let's stage it to Production:



Now it is in **Production**!!!



We can connect with the model from the notebook too and extract information:

```
# 4. Registration of model
model_name = 'SentimentClassifier'
model_mlflow = mlflow.pyfunc.load_model(f'models:/{model_name}/Production')
✓ 0.4s

Downloading artifacts: 100%|██████████| 5/5 [00:00<00:00, 72.64it/s]
```

<-- From notebook

Connecting MLflow with G. Generative AI and LangChain

From the notebook, we can also connect via API key directly to Google AI Studio. We already knew that we can connect to MLflow.
We just need to keep the API key safely out of view, in a **.env** file, and retrieve it with *os.getenv()*

```
import os
import mlflow
from dotenv import load_dotenv
✓ 0.0s                                                    Python

load_dotenv()

# Configure tracking URI
MLFLOW_TRACKING_URI = "http://127.0.0.1:8080"
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)

print(f"MLflow Tracking URI: {mlflow.get_tracking_uri()}")

# Configure the API key for Google
GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")

if not GOOGLE_API_KEY:
    raise ValueError("Configure GOOGLE_API_KEY in your .env file")

print("API key correctly configured.")
✓ 0.0s                                                    Python

MLflow Tracking URI: http://127.0.0.1:8080
API key correctly configured.
```

If we set MLflow to track Gemini, it will automatically log the calls we make from the notebook. First, we set up an experiment.

```python
import google.generativeai as genai

# Mobilize auto-tracking for Gemini
mlflow.gemini.autolog()

# Configure the experiment
mlflow.set_experiment("Gemini_Direct_Tracking")

# Configure Google SDK
experiment = genai.configure(api_key=GOOGLE_API_KEY)

print("MLflow Gemini autolog enabled.")
print(f"Experiment: {experiment}")
```

✓ 0.1s Python

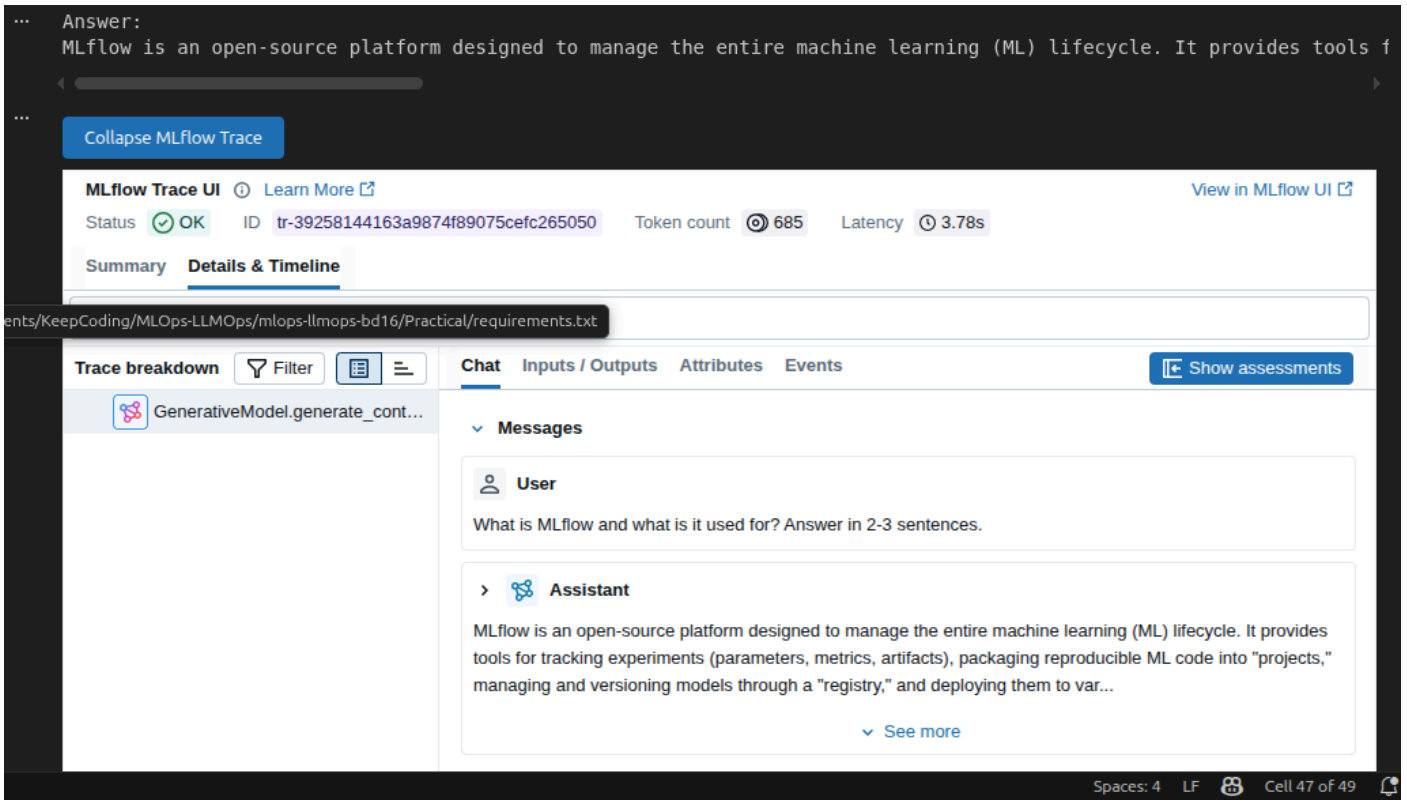
```
2026/03/03 19:57:41 INFO mlflow.tracking.fluent: Experiment with name 'Gemini_Direct_Tracking' does not exist. Creat
MLflow Gemini autolog enabled.
Experiment: <Experiment: artifact_location='mlflow-artifacts:/3', creation_time=1772564261753, experiment_id='3', la
```

Then we can make our first call: We define the generative model we want to use, and assign a variable to the output of our prompt (here: *response*), so that we can print it. As soon as we run the cell, an MLflow interactive user interface appears in the notebook, showing the experiment we just ran. (I find it easier to go to the MLflow API directly from my browser, though.)

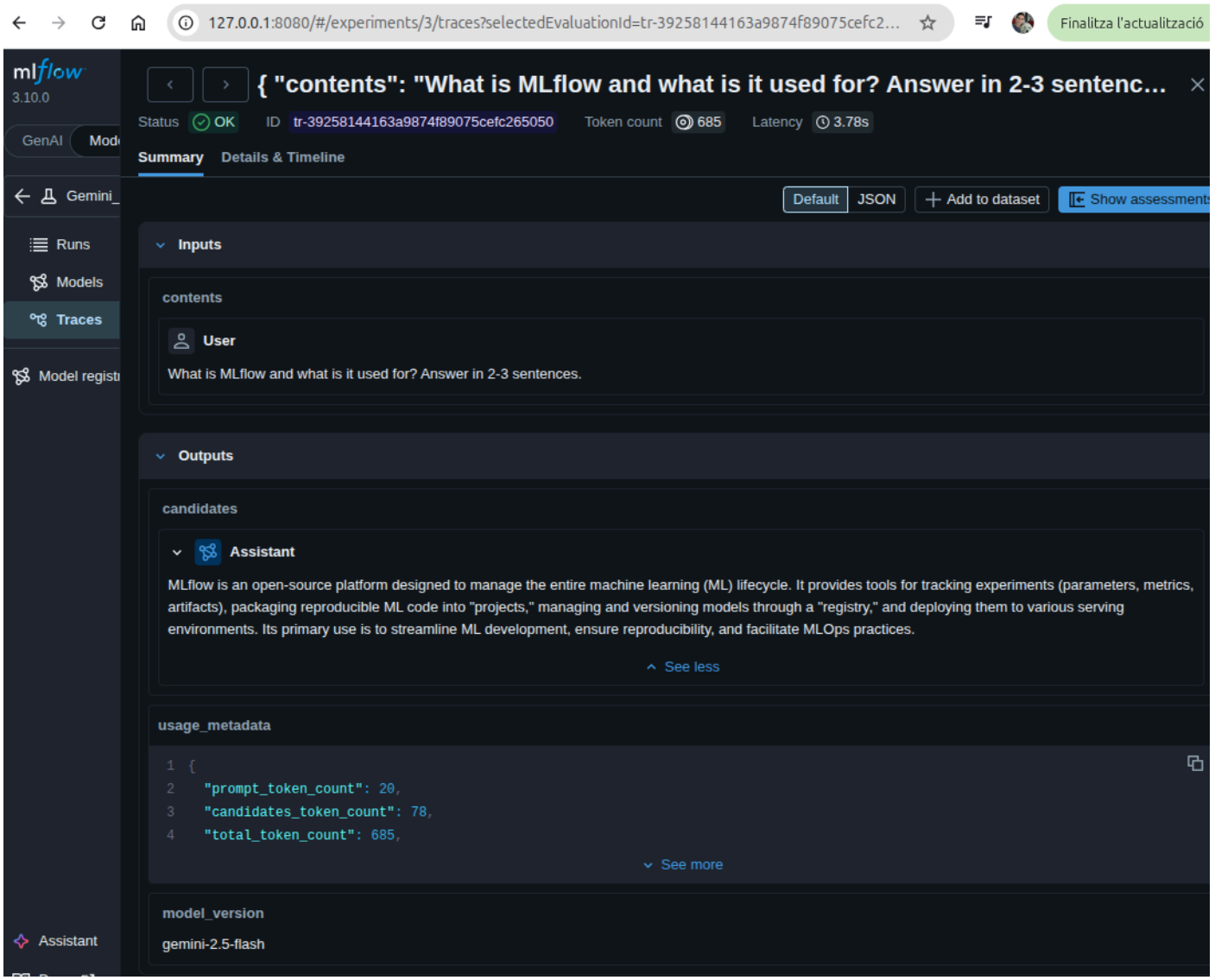
```python
model = genai.GenerativeModel("gemini-2.5-flash")

# Start with a simple call - MLflow will automatically capture the trace
response = model.generate_content("What is MLflow and what is it used for? Answer in 2-3 sentences.")

print("Answer: ")
print(response.text)
```

✓ 4.0s Python

```
Answer:
MLflow is an open-source platform designed to manage the entire machine learning (ML) lifecycle. It provides tools f
```



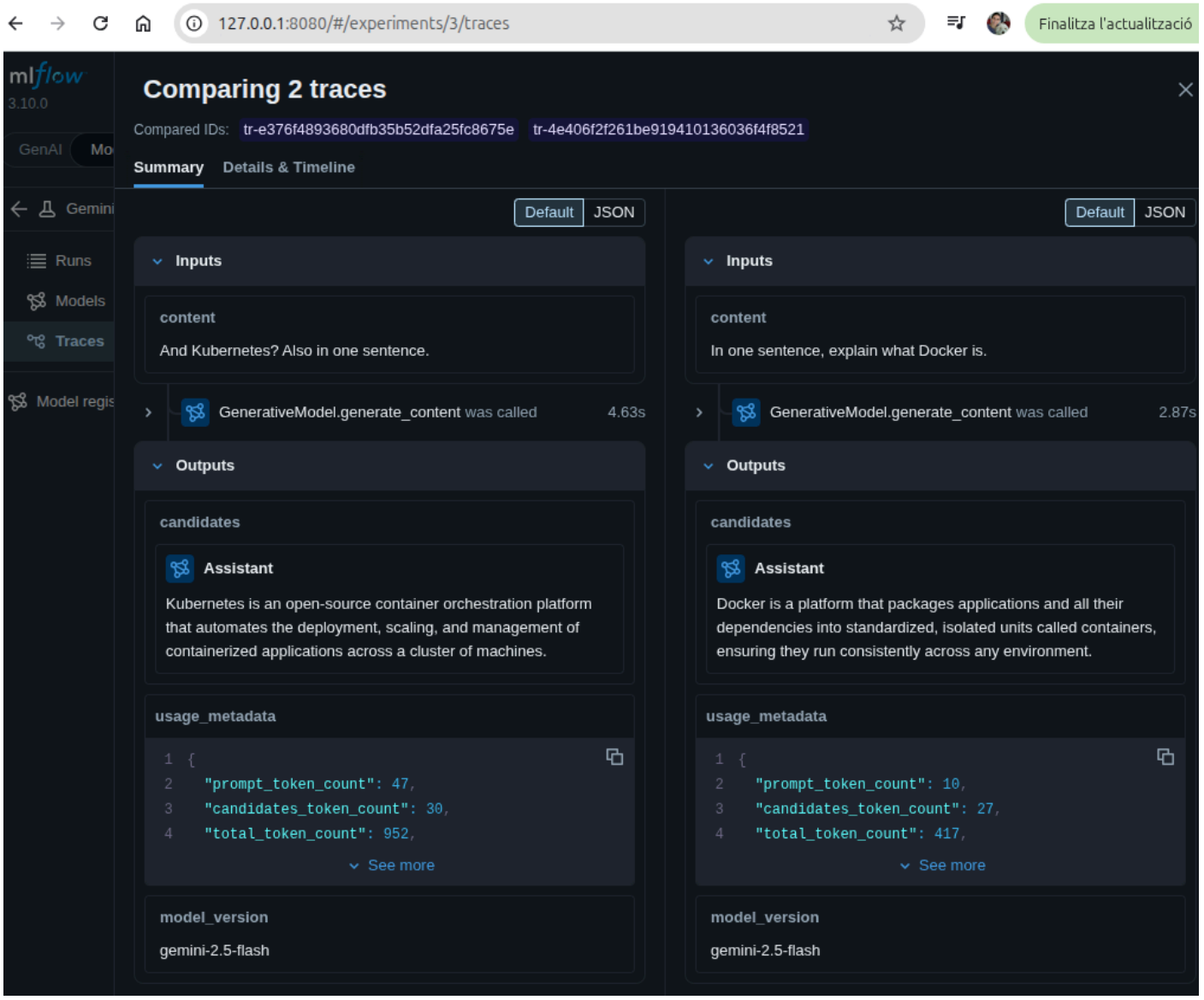
I prefer the view directly from the MLflow API (Click on the experiment, then on Traces):



It's interesting to be able to see the **token count** of both the prompt and the "candidates", which is the response. This all goes into our usage rate.

We can also send multiple prompts at once, and later make a **side-by-side comparison** in MLflow (I could also have added the earlier prompt to this comparison, and compare all three prompts.)

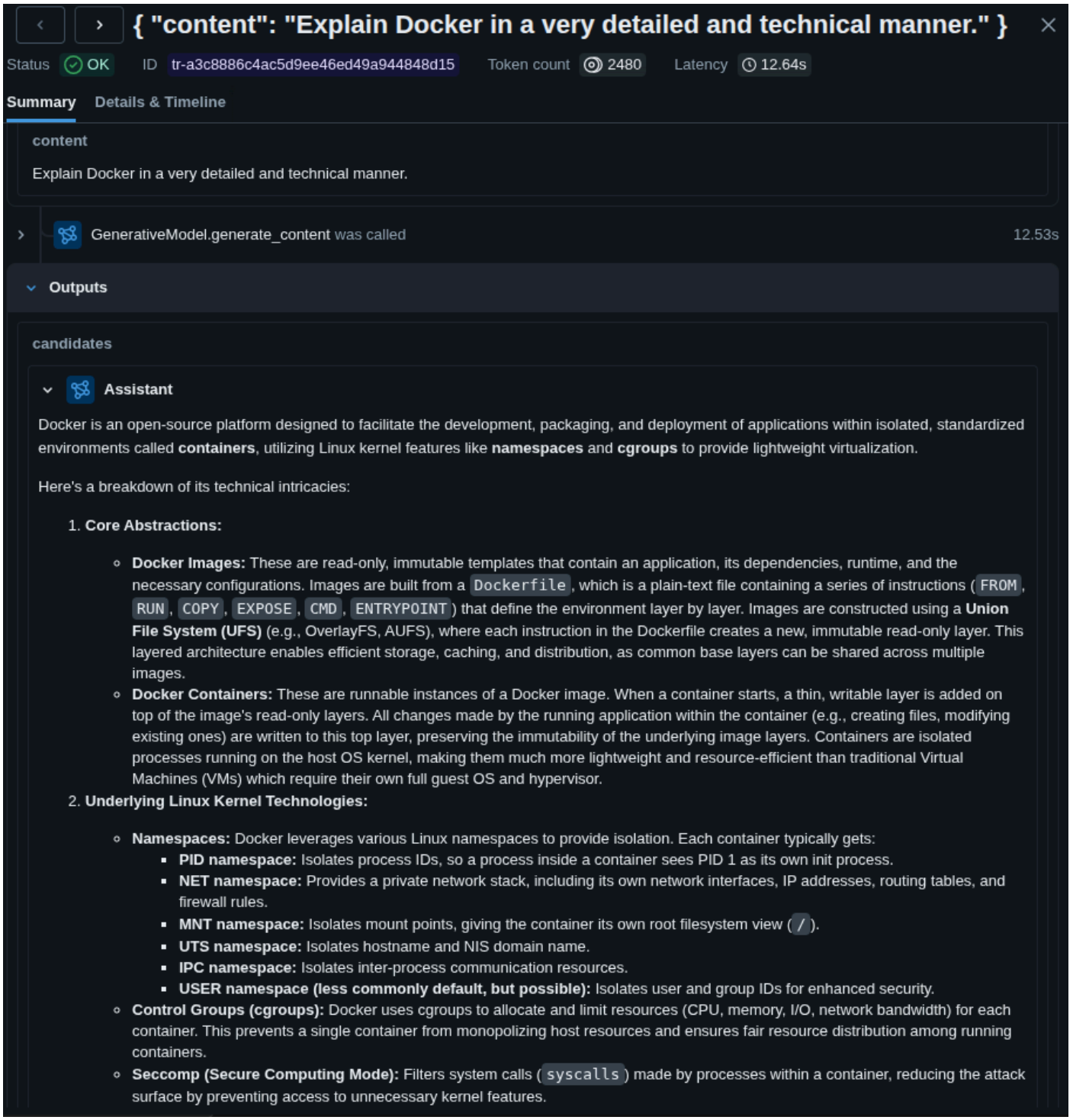
Here, it's interesting to see that the Kubernetes prompt used many more tokens than the Docker prompts, and also took longer (4.63 seconds versus 2.87 seconds).



We could also use such side-by-side comparisons to evaluate differences to the same prompt by different models, for example.

Or also to evaluate, for the same exact prompt, differences in response depth or response time, given different system roles.

In another example, I made a prompt, asking for technical details about Docker.
Below is an excerpt of the response:



{ "content": "Explain Docker in a very detailed and technical manner." }

Status OK ID tr-a3c8886c4ac5d9ee46ed49a944848d15 Token count 2480 Latency 12.64s

Summary Details & Timeline

content

Explain Docker in a very detailed and technical manner.

GenerativeModel.generate_content was called 12.53s

Outputs

candidates

Assistant

Docker is an open-source platform designed to facilitate the development, packaging, and deployment of applications within isolated, standardized environments called **containers**, utilizing Linux kernel features like **namespaces** and **cgroups** to provide lightweight virtualization.

Here's a breakdown of its technical intricacies:

1. **Core Abstractions:**
    - **Docker Images:** These are read-only, immutable templates that contain an application, its dependencies, runtime, and the necessary configurations. Images are built from a `Dockerfile`, which is a plain-text file containing a series of instructions (`FROM`, `RUN`, `COPY`, `EXPOSE`, `CMD`, `ENTRYPOINT`) that define the environment layer by layer. Images are constructed using a **Union File System (UFS)** (e.g., OverlayFS, AUFS), where each instruction in the Dockerfile creates a new, immutable read-only layer. This layered architecture enables efficient storage, caching, and distribution, as common base layers can be shared across multiple images.
    - **Docker Containers:** These are runnable instances of a Docker image. When a container starts, a thin, writable layer is added on top of the image's read-only layers. All changes made by the running application within the container (e.g., creating files, modifying existing ones) are written to this top layer, preserving the immutability of the underlying image layers. Containers are isolated processes running on the host OS kernel, making them much more lightweight and resource-efficient than traditional Virtual Machines (VMs) which require their own full guest OS and hypervisor.
2. **Underlying Linux Kernel Technologies:**
    - **Namespaces:** Docker leverages various Linux namespaces to provide isolation. Each container typically gets:
        - **PID namespace:** Isolates process IDs, so a process inside a container sees PID 1 as its own init process.
        - **NET namespace:** Provides a private network stack, including its own network interfaces, IP addresses, routing tables, and firewall rules.
        - **MNT namespace:** Isolates mount points, giving the container its own root filesystem view (`/`).
        - **UTS namespace:** Isolates hostname and NIS domain name.
        - **IPC namespace:** Isolates inter-process communication resources.
        - **USER namespace (less commonly default, but possible):** Isolates user and group IDs for enhanced security.
    - **Control Groups (cgroups):** Docker uses cgroups to allocate and limit resources (CPU, memory, I/O, network bandwidth) for each container. This prevents a single container from monopolizing host resources and ensures fair resource distribution among running containers.
    - **Seccomp (Secure Computing Mode):** Filters system calls (`syscalls`) made by processes within a container, reducing the attack surface by preventing access to unnecessary kernel features.

Comparing the number of tokens used in this technical question compared to the prior prompt, where I was just asking for an explanation in one sentence:

Comparing 2 traces

Compared IDs: tr-a3c8886c4ac5d9ee46ed49a944848d15 tr-4e406f2f261be919410136036f4f8521

Summary | Details & Timeline

- **Networking:** Docker provides various networking options. By default, containers connect to a virtual bridge network (`docker0`), allowing them to communicate with each other and the host, often through Network Address Translation (NAT) for outbound internet access. User-defined bridge networks, host-mode networking (container shares host's network stack), and overlay networks (for multi-host communication with tools like Docker Swarm or Kubernetes) offer more sophisticated configurations.
- **Persistent Storage:** Since the writable layer of a container is ephemeral, Docker offers mechanisms for persistent data storage:
  - **Volumes:** Docker-managed storage areas that are independent of the container's lifecycle. They are typically stored in a part of the host filesystem managed by Docker.
  - **Bind Mounts:** Allow mounting any directory or file from the host machine directly into a container, giving the container direct access to host data.
  - `tmpfs` **Mounts:** Store data in the host's memory, suitable for very temporary or sensitive non-persistent data.

In essence, Docker abstracts the underlying infrastructure, providing a consistent and isolated environment for applications across diverse computing environments, from development machines to production servers, by leveraging advanced Linux kernel features for lightweight virtualization and resource management.

See less

usage_metadata


```
{
  "prompt_token_count": 89,
  "candidates_token_count": 1097,
  "total_token_count": 2480,
  "cached_content_token_count": 0
```


Default | JSON

Inputs

content

In one sentence, explain what Docker is.

GenerativeModel.generate_content was called 2.87s

Outputs

candidates

Assistant

Docker is a platform that packages applications and all their dependencies into standardized, isolated units called containers, ensuring they run consistently across any environment.

usage_metadata


```
{
  "prompt_token_count": 10,
  "candidates_token_count": 27,
  "total_token_count": 417,
  "cached_content_token_count": 0
}
```


See less

model_version

gemini-2.5-flash

```python
prompt_versions = [
    {
        "name": "basic",
        "system": "You are a helpful assistant.",
        "user": "What is Docker?"
    },
    {
        "name": "expert",
        "system": "You are a DevOps expert with 10 years experience. Respond in a technical but accessible manner
        "user": "What is Docker?"
    },
    {
        "name": "structured",
        "system": "You are a DevOps expert. Respond in this format: 1) Definition, 2) Usage cases, 3) Benefits.",
        "user": "What is Docker?"
    }
]
for version in prompt_versions:
    print(f"\n{'='*50}")
    print(f"Version: {version['name']}")
    print(f"{'='*50}")

    messages = [
        SystemMessage(content=version["system"]),
        HumanMessage(content=version["user"])
    ]
    response = llm.invoke(messages)
    print(response.content[:300] + "...")
```

This final MLflow comparison comes from prompts made using the LangChain library, with the model “gemini-2.5-flash-lite”, and the loop shown above. Output, from left to right:

STRUCTURED
"system": "You are a DevOps expert. Respond in this format: 1) Definition, 2) Usage cases, 3) Benefits."
"user": "What is Docker?"

EXPERT
"system": "You are a DevOps expert with 10 years experience. Respond in a technical but accessible manner."
"user": "What is Docker?"

BASIC
"system": "You are a helpful assistant."
"user": "What is Docker?"

Comparing 3 traces

Compared IDs: tr-f01119081a155c585912b95eb2dfac94 tr-3286345c22cbb3fcdf1498003cd641f4 tr-157a516d03a508eac0ae8e8653f08f73

Summary | Details & Timeline

Here's a breakdown of Docker from a DevOps perspective:

1) Definition

Docker is an open-source platform that automates the deployment, scaling, and management of applications using **containers**. A container is a lightweight, standalone, executable package of software that includes everything needed to run it: code, runtime, system tools, system libraries, and settings. Docker containers virtualize the operating system, allowing applications to run in isolated environments while sharing the host OS kernel. This contrasts with traditional virtual machines (VMs) which virtualize the entire hardware stack and run a full operating system.

2) Usage Cases

Docker is widely adopted across various stages of the software development lifecycle and in production environments. Key usage cases include:

- **Development Environment Consistency:** Developers can package their application and its dependencies into a Docker image. This ensures that the application runs exactly the same way on any machine that has Docker installed, eliminating the "it works on my machine" problem.
- **CI/CD Pipelines:** Docker integrates seamlessly with Continuous Integration and Continuous Deployment (CI/CD) tools. Developers can build Docker images as part of the CI process, and these images can then be deployed to staging or production environments without modification.
- **Microservices Architecture:** Docker is ideal for deploying and managing microservices. Each microservice can be containerized independently, allowing for easier development, deployment, scaling, and updates of individual services without affecting others.

Alright, let's break down Docker from a DevOps perspective. Imagine you're building and deploying software, and you've probably run into the classic "it works on my machine" problem. You know, where your application runs perfectly on your laptop, but then when you try to run it on a staging server or in production, it breaks spectacularly because of some subtle difference in the operating system, libraries, or configuration.

Docker is essentially a tool that solves this problem by packaging your application and all its dependencies (libraries, system tools, code, runtime – you name it) into a standardized, self-contained unit called a "container."

Think of it like this:

- **Traditional Deployment (Without Docker):** You're building a house. You have the blueprints (your code), but you need to go to different lumber yards, hardware stores, and specialty shops to get all the materials (dependencies). Then, you have to assemble everything on-site, hoping the ground conditions and local building codes are exactly what you expected. If something's missing or different at the construction site, the whole project can stall.
- **Docker Deployment:** You're now using pre-fabricated modules for your house. Each module (a Docker container) is built in a controlled factory environment. It contains *everything* needed to function: the walls, the plumbing, the electrical wiring, the appliances – all perfectly integrated. When you get to the construction site, you just slot these modules together. They're designed to fit and work regardless of minor variations in the "site" (the server).

Key Concepts in Docker:

Docker is an open-source platform that **automates the deployment, scaling, and management of applications using containers.**

Think of containers as **lightweight, standalone, executable packages of software** that include everything needed to run an application: the code, runtime, system tools, system libraries, and settings.

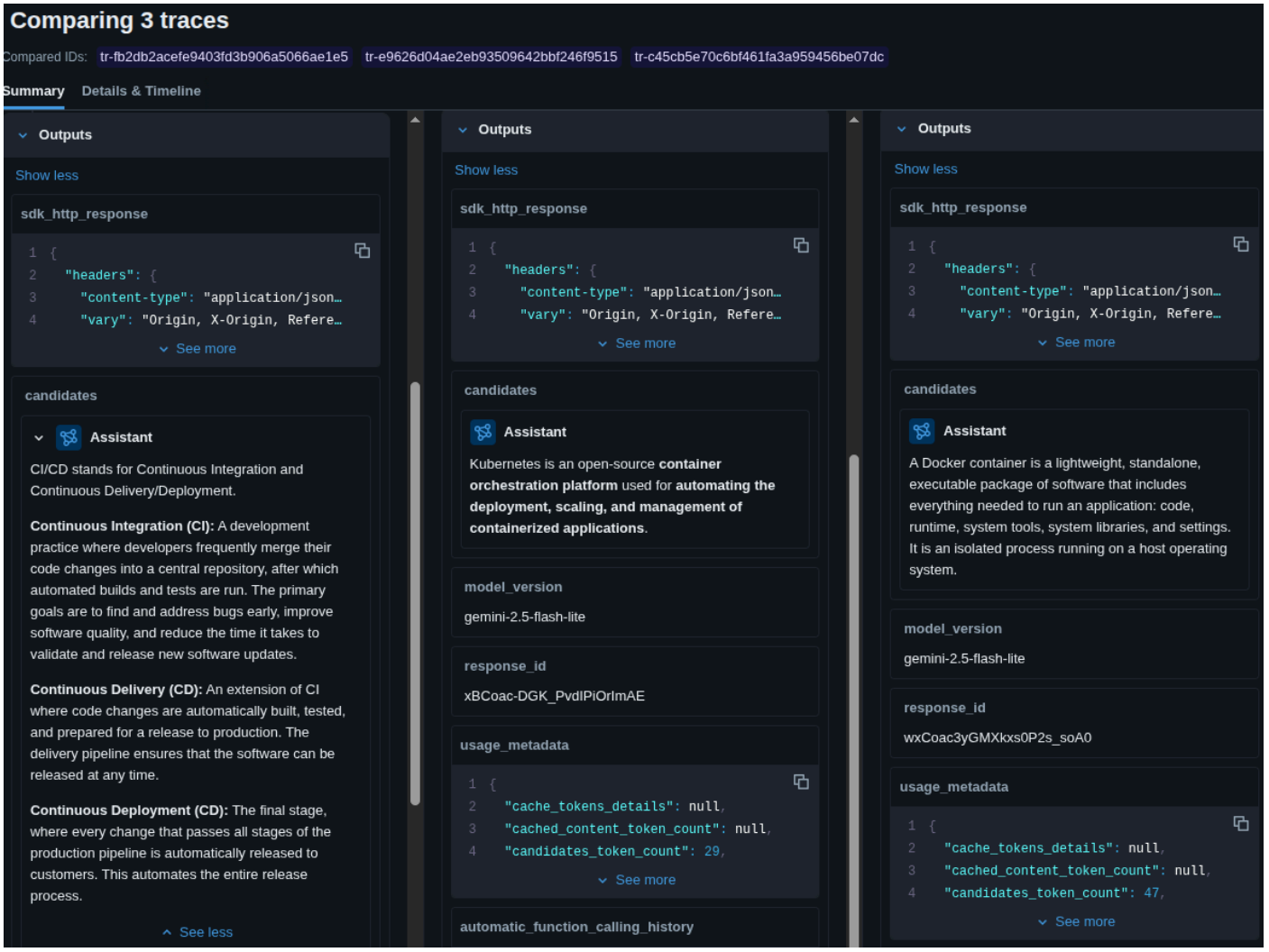
Here's a breakdown of what that means and why it's so powerful:

Key Concepts:

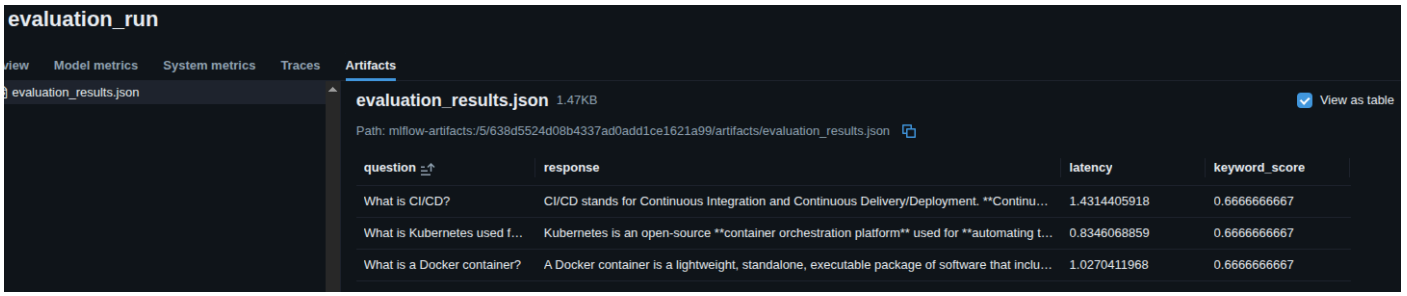
- **Containers:**
  - **Isolation:** Containers isolate applications from each other and from the underlying host system. This means an application running in one container won't interfere with another, and it won't mess up your host's environment.
  - **Portability:** Because containers package everything an application needs, they can run consistently across different environments – from a developer's laptop to a testing server to a production cloud. "Build once, run anywhere."
  - **Lightweight:** Unlike virtual machines (VMs) that virtualize an entire operating system, containers share the host OS kernel. This makes them much smaller, faster to start up, and more resource-efficient.
- **Docker Engine:** This is the core of Docker. It's a client-server application that includes:
  - **Docker Daemon:** A background process that manages Docker objects like images, containers, networks, and volumes.
  - **Docker CLI (Command Line Interface):** The tool you use to interact with the Docker daemon (e.g., `docker build`, `docker run`, `docker ps`).

In this final example, we included an evaluation for each of the prompts. Below are the questions and responses:

What is CI/CD? What is Kubernetes used for? What is a Docker container?



And these are the results of the evaluation, based on the expected keywords defined in the code:

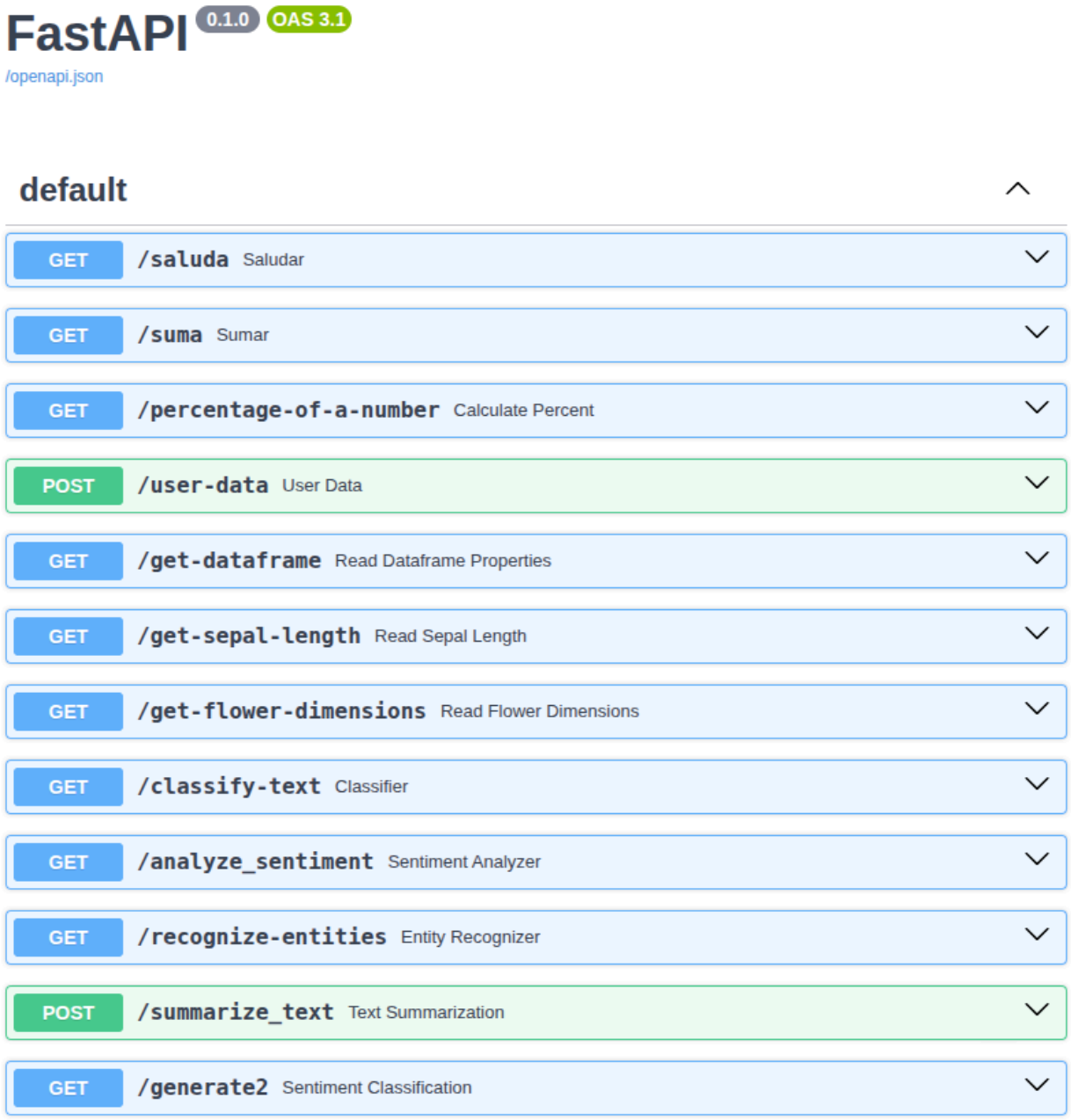


Metric	Value
latency_q0	1.0270411968231201
keyword_score_q0	0.6666666666666666
latency_q1	0.8346068859100342
keyword_score_q1	0.6666666666666666
latency_q2	1.4314405918121338
keyword_score_q2	0.6666666666666666
avg_latency	1.0976962248484294
avg_keyword_score	0.6666666666666666

2. API Screenshots

The actual scripts are located in the file hello_fastapi.py

Screenshot of the different modules:



The last 5 modules in the list above are pipelines from HuggingFace. But I could only get the first two, /classify-text and /analyze_sentiment, to work properly.

Since they are essentially doing the same thing, I set one up with the prompt already written in the script (as a list of several excerpts to analyze) and the other where the prompt is to be inserted from FastAPI.

Screenshot of each module :

GET /saluda Saludar

Parameters

Cancel

Name	Description
name * required string (query)	Patti

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/saluda?name=Patti' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/saluda?name=Patti
```

Server response

Code	Details
200	Response body "Hello, Patti. I hope your day is as beautiful as you are :)" Download Response headers content-length: 61 content-type: application/json date: Thu,05 Mar 2026 00:00:05 GMT server: uvicorn

Responses

Code	Description	Links

GET /suma Sumar

Parameters

Cancel

Name	Description
arg1 * required number (query)	2.35
arg2 * required number (query)	-11.28

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/suma?arg1=2.35&arg2=-11.28' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/suma?arg1=2.35&arg2=-11.28
```

Server response

Code	Details
200	Response body -8.93 Download Response headers

GET /percentage-of-a-number Calculate Percent

Parameters

Cancel

Name	Description
percent * required number (query)	28
value * required number (query)	2135

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/percentage-of-a-number?percent=28&value=2135' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/percentage-of-a-number?percent=28&value=2135
```

Server response

Code	Details
200	Response body "28.0% of 2135.0 = 597.8" Download Response headers

POST /user-data User Data

Parameters

Cancel Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "name": "Ari",
  "surname": "Princess",
  "second_surname": "Warrior",
  "phone_number": "586 708 234"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/user-data' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "Ari",
  "surname": "Princess",
  "second_surname": "Warrior",
  "phone_number": "586 708 234"
}'
```

GET /get-dataframe Read Dataframe Properties

Parameters

Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/get-dataframe' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/get-dataframe
```

Server response

Code	Details
200	Response body


```
{
  "sepal_length": {
    "0": 5.1,
    "1": 4.9,
    "2": 4.7,
    "3": 4.6,
    "4": 5,
    "5": 5.4,
    "6": 4.6,
    "7": 5,
    "8": 4.4,
    "9": 4.9,
    "10": 5.4,
    "11": 4.8,
    "12": 4.8,
    "13": 4.3,
    "14": 5.8,
    "15": 5.7,
    "16": 5.4,
    "17": 5.1,
    "18": 5.7,
    "19": 5.1,
    "20": 5.4,
    "21": 5.1,
    "22": 4.6,
```

GET /get-sepal-length Read Sepal Length

Parameters

Cancel

Name	Description
position * required integer (query)	91

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/get-sepal-length?position=91' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/get-sepal-length?position=91
```

Server response

Code	Details
200	Response body 6.1 Download Response headers content-length: 3 content-type: application/json date: Thu,05 Mar 2026 01:23:07 GMT server: uvicorn

Responses

Code	Description	Links

GET /get-flower-dimensions Read Flower Dimensions

Parameters

Cancel

Name	Description
position * required integer (query)	79

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/get-flower-dimensions?position=79' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/get-flower-dimensions?position=79
```

Server response

Code	Details
200	Response body "Sepal length = 5.7 Sepal width = 2.6 Petal length = 3.5" Download Response headers content-length: 65 content-type: application/json date: Thu,05 Mar 2026 00:07:37 GMT server: uvicorn

Responses

Code	Description	Links

Pipelines (Transformers):

GET /classify-text Classifier

Parameters

Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/classify-text' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/classify-text
```

Server response

Code	Details
200	Response body


```
[
  {
    "label": "POSITIVE",
    "score": 0.9998724460601807
  },
  {
    "label": "NEGATIVE",
    "score": 0.8038442730903625
  },
  {
    "label": "POSITIVE",
    "score": 0.9991185069084167
  },
  {
    "label": "POSITIVE",
    "score": 0.9995982050895691
  },
  {
    "label": "NEGATIVE",
    "score": 0.7884781956672668
  }
]
```


Download

Response headers

Prompts:

['Amazing park, with lots of weird creatures!',
'Nice setting, but the food was terrible',
'Not bad at all',
'This was a blast',
'If you care about your health, avoid at all costs']

GET /analyze_sentiment Sentiment Analyzer

Parameters

Cancel

Name	Description
text * required any (query)	How did we get to this level of conflict in the \

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/analyze_sentiment?text=How%20did%20we%20get%20to%20this%20level%20of%20confl
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/analyze_sentiment?
text=How%20did%20we%20get%20to%20this%20level%20of%20conflict%20in%20the%20world%3F
```

Server response

Code	Details
200	Response body

```
[
  {
    "label": "NEGATIVE",
    "score": 0.996141254901886
  }
]
```

Download

Response headers

```
content-length: 48
content-type: application/json
date: Thu,05 Mar 2026 00:49:16 GMT
server: uvicorn
```

Responses

Prompt: ‘How did we get to this level of conflict in the world?”